

1. Introduction

This system simulates a single airport's runway operations, manages aircraft arrivals and departures while maintaining constraints like runway occupancy, altitude separation and fuel limitations. The simulation runs in scaled time, with the timescale being configurable by the user, allowing for rapid testing and real-time emulation. The main operations handled by the system are:

1. **Aircraft arrival & departure scheduling** - aircraft are scheduled based on generated flight data.
2. **Runway assignment** - Available runways are dynamically allocated based on occupancy status and operating mode
3. **Emergency prioritisation** - Aircraft which have declared emergencies are handled before standard traffic
4. **Fuel monitoring and diversions** - Aircraft's fuel levels are tracked in real time, and aircraft which fall below a certain threshold are diverted if a runway cannot be assigned in time.
5. **Simulation statistics tracking** - Key metrics like delay, diversions and queue sizes are recorded throughout the simulation to be displayed in both live time and for post-run analysis.

2. System Architecture

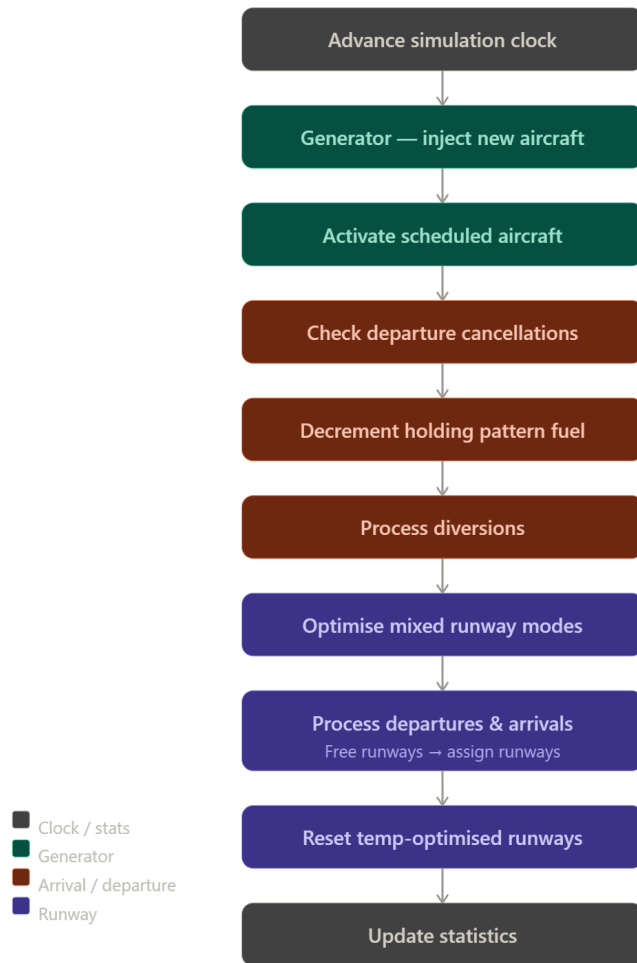
The backend follows a modular controller architecture, with a central controller managing the simulation time and global tick, co-ordinating each individual component of the system by calling them in a fixed order each tick. Each controller operates independently and communicates with others through a shared Django database layer.

The backend is split into the following components.

2.1 Controller

Orchestrator of the simulation. Reads RunConfig from the database and uses it to configure all subsystems. It then calls `run_simulation()` on every tick, which performs the following action in a fixed order:

1. Advances the simulation clock based on real elapsed time and the configured timescale
2. Run Generator to inject newly scheduled flights
3. Move scheduled aircraft into their respective queues based on queue entry time
4. Checks for departure cancellations (flights that have waited in the queue for too long)
5. Decrements fuel for aircraft in holding pattern
6. Checks and processes any diversions
7. Optimises mixed runway modes
8. Processes departures and arrivals
9. Resets temporarily optimised runways back to Mixed
10. Logs the min/max queue size



2.2 ArrivalController

Manages all inbound traffic. The key responsibilities of the ArrivalController include:

- **Fuel decrementing (`update_aircraft_fuel`)** - called every simulation minute to reduce `fuel_mins` by 1 for all aircraft currently in the holding pattern
- **Handling diversions (`update_aircraft_diversions`)** - Aircraft with 10 or fewer minutes of fuel remaining are marked as diverted and removed from the stack and added to the maximum number of planes diverted.
- **Altitude recalculation (`recalculate_altitudes`)** - After any changes to the stack (a new arrival, landing or diversion) altitudes are reassigned from 1000ft in 1000ft increments. Aircraft with emergencies are placed at the bottom of this stack.
- **Processing landings (`process_arrivals`)** - Frees runways for aircraft which have completed their landing (based on a variable landing duration) and assigns available runways to aircrafts in queue. Emergencies are processed with priority over normal traffic

2.3 DepartureController

Manages all outbound traffic. The key responsibilities include:

- **Cancellation handling (update_aircraft_cancellations)** - Marks aircraft which have been in the departure queue longer than the maximum wait time as cancelled.
- **Departure processing (process_departures)** - Frees runways for aircraft which have completed taking off (based on a variable take off duration) and then assigns available runways to aircrafts in the departure queue in FIFO order.

2.4 RunwayController

Interface between aircraft and runways, with responsibilities including:

- **Runway assignment (assign_runway)** - Finds an available runway whose operating mode matches the aircraft (landing/taking off). Marks the runway as occupied and links it to the aircraft
- **Runway freeing (free_runway)** - Checks if an aircraft has occupied the runway for the required duration. If it has, then the runway is freed and an aircraft's status is moved to LANDED/DEPARTED.
- **Runway Optimisation (optimise_runway_mode)** - Called at every tick to temporarily re-assign mixed runways to landing/take-off only based on current traffic. The priority order is emergencies, then diversion/cancellation risks and then whichever queue is larger (holding pattern/take-off queue)

2.5 Generator

Responsible for populating the simulation with new aircraft objects. On each call of `run_generation()`, it generates inbound and outbound flights until the schedule window (simulation time + hour_view) is filled. Each aircraft object is built from a random row of a real-world flight CSV dataset. A normally distributed delay is applied to the scheduled time (mean 0, s.d. 5 minutes) and a 1% chance exists per aircraft for a random emergency (medical or mechanical) to be assigned to it.

2.6 Data Layer

The system uses Django Models as a centralised state repository, allowing controllers to 'communicate' by modifying and reading the states of objects in the database as opposed to directly sending data to one another.

2.6.1 Aircraft Model

Key fields:

- **callsign** - flight identifier
- **scheduled_arrival/scheduled_departure** - original scheduled time
- **queue_entry_time** - the time the aircraft is expected to enter the queue (scheduled time + delay)

- **zone_status** - aircraft's current phase (SCHEDULED, QUEUE_LA, QUEUE_TO, RUNWAY_LA, RUNWAY_TO, LANDED, DEPARTED, CANCELLED, DIVERTED)
- **assigned_runway** - Foreign key to the runway the aircraft is using (if any)
- **altitude** - current holding stack altitude in feet, assigned by ArrivalController
- **fuel_mins** - remaining fuel in minutes
- **emergency_status** - one of NONE, FUEL, MECHANICAL or MEDICAL.
- **last_update** - timestamp used by update_aircraft_fuel to track when fuel was last decremented

2.6.2 Runway Model

Key fields:

- **bearing** - runway heading in degrees
- **length** - runway length in metres
- **operating_mode** - one of LANDING, TAKEOFF or MIXED
- **operational_status** - one of AVAILABLE, OCCUPIED, RUNWAYINSPEC, SNOWCLEAR or EQUIPFAIL. Only AVAILABLE runways can be assigned
- **occupied_by** - foreign key to the Aircraft currently on the runway
- **time_occupied** - the simulation time at which the aircraft began using the runway
- **temp_optimised** - boolean flag to indicate if runways mode was temporarily changed from mixed

2.6.3 RunConfig Model

Stores the configurable parameters for a simulation run. All controllers read from this on initialisation, and update_configuration() can re-apply it mid-run. Key fields include timescale, inbound_per_hour, outbound_per_hour, landing_duration, takeoff_duration, max_wait, fuel_risk_threshold, takeoff_risk_threshold, and schedule_limit.

2.6.4 RunStats Model

A single record model which accumulates statistics throughout the simulation. Tracks running sums, means and variances for holding time, takeoff queue wait time, arrival delay and departure delay as well as peak and minimum values for queue sizes, delays, cancellations and diversions.

3. API & Backend communication

The frontend communicates with the Django backend through a combination of REST API endpoints and a Server-Sent Events (SSE) stream.

The SSE stream at /stream/ is the core of live simulation view, the frontend connects with it once and receives a continuous feed of flight and time data per tick. All other interactions use standard REST calls

Key endpoints

GET /stream/	Live simulation feed (SSE)
--------------	----------------------------

GET /api/runway-data/	Current state of all runways
GET /api/sample-data/	Preview scheduled flights
GET /api/end-simulation/	Stop the simulation
POST /api/close-runway/	Close a runway
POST /api/open-runway/	Re-open a runway
POST /api/change-timescale/	Adjust simulation speed mid-run

4. Deployment

The system is deployed and can be accessed at <https://web-production-16b0a.up.railway.app/> without the need for local hosting. The application is hosted on the Railway cloud platform. The deployment is connected to our **GitHub** repository, so whenever a new commit is pushed to the main branch, Railway automatically detects the update and starts a new deployment. As part of this process, the platform rebuilds the application, applies any necessary database migrations and prepares the static files before launching the updated version of the system.

5. Limitations and Assumptions

- Our simulation only models one airport, so diversions treat the aircraft as having left the entire system rather than routing it to another airport
- No weather system, factors like wind direction are not considered
- Fuel is decremented at a constant rate of 1 per minute, real consumption rates will vary
- No taxi time/ground movement is considered, once an aircraft lands or departs it is removed from the runway once the occupancy duration ends
- Runway bearing is generated randomly and not used in any way which affects runway assignment logic.